

MATERIAL DE APOIO PARA AULA

NoSQL com Aggregation Pipeline

Aggregate com lookup condicionais e facet no MongoDB

Este material apresenta uma aula prática sobre Aggregation Pipeline no MongoDB. A turma aprenderá a relacionar coleções, aplicar regras de classificação e gerar várias análises em uma única consulta. Todo o conteúdo foi organizado em elementos editáveis para permitir adaptações, observações e novos exemplos.

Resultado esperado: Ao final, os estudantes deverão conseguir ler, explicar e montar uma pipeline que combine \$lookup, \$switch ou \$cond e \$facet.

1 Objetivos e roteiro

Objetivos de aprendizagem

- Reconhecer a estrutura de uma Aggregation Pipeline.
- Relacionar coleções com \$lookup e tratar o resultado com \$unwind.
- Aplicar regras de negócio com \$switch ou \$cond, equivalentes a um CASE.
- Produzir múltiplos relatórios simultaneamente com \$facet.
- Ler e explicar uma pipeline que combina os três recursos.

Ideia central: Uma pipeline é uma sequência de etapas. Cada estágio recebe documentos, transformos e entrega o resultado ao próximo estágio.

2 Cenário da aula

Usaremos uma pequena loja com duas coleções: clientes e pedidos. O campo `clienteId` em pedidos referencia o campo `_id` em clientes.

Coleção clientes

```
db.clientes.insertMany([
  { _id: 1, nome: "Ana", cidade: "São Paulo" },
  { _id: 2, nome: "Bruno", cidade: "Campinas" },
  { _id: 3, nome: "Carla", cidade: "São Paulo" }
]);
```

Coleção pedidos

```
db.pedidos.insertMany([
  { _id: 101, clienteId: 1, valor: 250,
    status: "pago", data: ISODate("2026-09-10") },
  { _id: 102, clienteId: 2, valor: 80,
    status: "pendente", data: ISODate("2026-09-11") },
  { _id: 103, clienteId: 1, valor: 650,
    status: "pago", data: ISODate("2026-09-12") },
  { _id: 104, clienteId: 3, valor: 1400,
    status: "pago", data: ISODate("2026-09-13") },
  { _id: 105, clienteId: 3, valor: 120,
    status: "cancelado", data: ISODate("2026-09-14") }
]);
```

Quais informações conseguimos obter somente com pedidos? Quais exigem dados da coleção clientes?

3 Fundamentos de aggregate

A função `aggregate` recebe uma lista de estágios. A ordem importa: filtrar antes de agrupar, por exemplo, reduz a quantidade de documentos processados nas etapas seguintes.

```
db.pedidos.aggregate([
  { $match: { status: "pago" } },
  {
    $group: {
      _id: "$status",
      quantidade: { $sum: 1 },
      totalVendido: { $sum: "$valor" },
      ticketMedio: { $avg: "$valor" }
    }
  }
]);
```

Como ler essa pipeline

- `$match` mantém apenas pedidos com status pago.
- `$group` reúne os documentos pelo campo status.
- `$sum: 1` conta documentos e `$sum: "$valor"` soma valores.
- `$avg` calcula a média dos valores do grupo.

Boa prática: Coloque filtros seletivos no início da pipeline sempre que isso preservar o resultado desejado.

4 Lookup relacionando coleções

O **\$lookup** realiza uma junção entre coleções. Ele adiciona um campo do tipo array, mesmo quando há somente um documento correspondente.

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "clienteId",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  {
    $project: {
      _id: 1,
      valor: 1,
      status: 1,
      cliente: "$cliente.nome",
      cidade: "$cliente.cidade"
    }
  }
]);
```

Parâmetros essenciais

Parâmetro	Função
from	Coleção que será consultada
localField	Campo da coleção atual
foreignField	Campo correspondente na outra coleção
as	Nome do array que receberá os documentos encontrados

Atenção: Sem \$unwind, cliente permanece como array.

5 Condicionais com **switch e cond**

O MongoDB não possui um operador chamado CASE. Para várias alternativas, usamos \$switch. Para uma decisão binária, usamos \$cond.

Classificação com switch

```
db.pedidos.aggregate([
  {
    $addFields: {
      faixaValor: {
        $switch: {
          branches: [
            { case: { $lt: ["$valor", 200] }, then: "Baixo" },
            { case: { $lt: ["$valor", 1000] }, then: "Médio" }
          ],
          default: "Alto"
        }
      }
    }
  }
]);
```

As condições são avaliadas na ordem. Um valor de 80 entra na primeira faixa; 650 entra na segunda; 1400 chega ao valor padrão.

Decisão simples com cond

```
{
  $addFields: {
    prioridade: {
      $cond: [
        { $gte: ["$valor", 1000] },
        "Atendimento especial",
        "Fluxo normal"
      ]
    }
  }
}
```

Erro comum: Ordenar as faixas de forma incorreta. Teste primeiro a condição mais específica ou construa intervalos explícitos com \$and.

6 Facet criando vários relatórios

O \$facet envia o mesmo conjunto de documentos para subpipelines independentes. O resultado é um único documento, com um array para cada visão produzida.

```
db.pedidos.aggregate([
  {
    $facet: {
      totalPorStatus: [
        {
          $group: {
            _id: "$status",
            quantidade: { $sum: 1 },
            total: { $sum: "$valor" }
          }
        },
        { $sort: { total: -1 } }
      ],
      pedidosAltos: [
        { $match: { valor: { $gte: 1000 } } },
        { $project: { _id: 1, clienteId: 1, valor: 1 } }
      ],
      resumoGeral: [
        {
          $group: {
            _id: null,
            quantidade: { $sum: 1 },
            media: { $avg: "$valor" }
          }
        }
      ]
    }
  }
]);
```

Formato conceitual do resultado

```
{
  totalPorStatus: [ ... ],
  pedidosAltos: [ ... ],
  resumoGeral: [ ... ]
}
```

Quando usar: Dashboards, páginas de busca com contagens por categoria e relatórios que precisam compartilhar o mesmo conjunto inicial de dados.

7 Pipeline completa

Nesta pipeline, os dados dos clientes são relacionados aos pedidos, uma categoria é calculada e três visões são produzidas sobre o mesmo conjunto enriquecido.

```
db.pedidos.aggregate([
  {
    $lookup: {
      from: "clientes",
      localField: "clienteId",
      foreignField: "_id",
      as: "cliente"
    }
  },
  { $unwind: "$cliente" },
  {
    $addFields: {
      categoria: {
        $switch: {
          branches: [
            { case: { $lt: ["$valor", 200] }, then: "Baixo" },
            { case: { $lt: ["$valor", 1000] }, then: "Médio" }
          ],
          default: "Alto"
        }
      }
    }
  },
  {
    $facet: {
      vendasPorCidade: [
        {
          $group: {
            _id: "$cliente.cidade",
            quantidade: { $sum: 1 },
            total: { $sum: "$valor" }
          }
        },
        { $sort: { total: -1 } }
      ],
      vendasPorCategoria: [
        {
          $group: {
            _id: "$categoria",
            quantidade: { $sum: 1 },
            total: { $sum: "$valor" }
          }
        }
      ],
      pedidosDetalhados: [
        {
          $project: {
            _id: 1,
            cliente: "$cliente.nome",
            cidade: "$cliente.cidade",
            valor: 1,
            status: 1,
            categoria: 1
          }
        },
        { $sort: { valor: -1 } }
      ]
    }
  }
]);
```

Fluxo mental

Etapa	Operador	Efeito
1	\$lookup	Acrescenta os dados do cliente
2	\$unwind	Transforma o array cliente em objeto
3	\$addFields	Cria a categoria por faixa de valor
4	\$facet	Divide o resultado em três relatórios

8 Atividade prática

Crie uma pipeline que produza um relatório de clientes e pedidos com os seguintes requisitos:

- Relacionar pedidos e clientes usando \$lookup.
- Agrupar por cliente e calcular totalGasto e quantidadePedidos.
- Classificar cada cliente como Bronze, Prata ou Ouro de acordo com o total gasto.
- Usar \$facet para retornar totais por cidade, quantidade por categoria e uma lista detalhada de clientes.

Regras de classificação

Categoria	Total gasto
Bronze	Menos de 500
Prata	De 500 a menos de 1500
Ouro	1500 ou mais

Desafio adicional

Inclua apenas pedidos pagos no cálculo. Pense em qual posição da pipeline o \$match deve ocupar.

O agrupamento por cliente deve ocorrer antes da classificação, pois a categoria depende do total acumulado, não do valor de um único pedido.

